

TatvaOS Mail API

Integration guide for developers — send mail from your own software, as your own organisation.

What this is

One HTTPS request that sends an email as one of your organisation's mailboxes. TatvaOS signs it (DKIM), delivers it from your organisation's verified domain (SPF, DMARC), and keeps a record of every send. You do not run a mail server, manage keys for DKIM, or deal with SMTP — you make a POST.

It is deliberately small. There is one endpoint, one authentication method, and six fields. A developer who has used any transactional mail API (Resend, Postmark, SendGrid) will recognise the shape.

At a glance

Endpoint	POST <code>https://core.tatvaos.com/api/v1/mail/send</code>
Authentication	Authorization: Bearer <code>tvos_...</code> — an organisation API key
Body	JSON: <code>from</code> , <code>to</code> , <code>subject</code> , <code>text</code> , <code>html</code> , <code>replyTo</code>
Recipients	Up to 5 per request, comma-separated in <code>to</code>
Sender	Must be a mailbox in your organisation, on a domain you have verified
Success	202 Accepted — accepted for delivery by TatvaOS

1. Getting an API key

An organisation administrator creates keys in the TatvaOS admin: **Organisation** → **API keys**. The administrator names the key for the program that will use it ("Website contact form", "Billing notifications") and hands you the key. It looks like this:

```
tvos_k7Qm2ZpR4wN9tV1cH6xB8sL3yD5fG0jA
```

Three things to know about it:

It is shown once. TatvaOS stores only a hash. If it is lost, the administrator revokes it and creates another; nobody can read it back.

It belongs to the organisation, but only to the senders chosen for it. When the key is created the administrator picks the addresses it may send from, and every active key has at least one — there is no key that can send as any mailbox. Choose which of those addresses to send as, per request, with the `from` field; anything outside the list is refused. See **Key restrictions** below.

It can send and nothing else. A key cannot read mail, list mailboxes, or change settings. If it leaks, the worst outcome is unwanted mail from your domain — which is why you still keep it secret.

Keep it server-side

Never put the key in browser JavaScript, a mobile app, or a public repository. Read it from an environment variable or a secrets store on your server, and call the API from there. If a key is ever exposed, ask the administrator to revoke it — revocation takes effect on the next request.

2. Choosing the sender

The `from` address must be an active mailbox in your organisation, on a domain the administrator has verified under **Organisation** → **Domains**. This is what lets TatvaOS sign the message and pass SPF and DMARC at

the receiving end — mail from an unverified domain would be refused or junked by every major provider, so TatvaOS refuses it first, with a clear message.

For software, use a **shared mailbox** (for example `website@`, `noreply@`, `billing@`). Replies then reach a team rather than one person's inbox. The administrator creates these under **Organisation** → **Shared mailboxes**.

If you want replies to go somewhere other than the sender, set `replyTo`.

Key restrictions

A key may send only from the addresses the administrator chose for it, and an active key always has at least one: there is no unrestricted key. A request whose `from` is outside that list is refused with a 400, and the error names the addresses the key is allowed to use — so the response tells you how to fix it.

Use this to separate keys by purpose: one for alerts, one for billing, one for the contact form. If a key leaks, only that sender is exposed, and revoking it stops that one program instead of all of them.

If a key has no allowed addresses it cannot send at all — an empty list means “nothing”, never “anything”. Ask your administrator to add the addresses the key should send from.

3. The request

One POST, JSON body, key in the Authorization header. Everything is UTF-8.

```
POST /api/v1/mail/send HTTP/1.1
Host: core.tatvaos.com
Authorization: Bearer tvos_k7Qm2ZpR4wN9tV1cH6xB8sL3yD5fG0jA
Content-Type: application/json
```

```
{
  "from": "website@your-domain.com",
  "to": "customer@example.com",
  "subject": "Thanks for getting in touch",
  "text": "We received your message and will reply within one working day.",
  "html": "<p>We received your message and will reply within <b>one working day</b>.</p>",
  "replyTo": "support@your-domain.com"
}
```

Fields

Field	Required	Meaning
<code>from</code>	yes	A mailbox in your organisation, on a verified domain. Plain address, no display name.
<code>to</code>	yes	One address, or up to five separated by commas: " <code>a@x.com</code> , <code>b@y.com</code> ". Each recipient gets their own copy and their own log entry.
<code>subject</code>	yes	The subject line.
<code>text</code>	one of	Plain-text body. Provide <code>text</code> , <code>html</code> , or both — both is best; the receiver's client picks.
<code>html</code>	one of	HTML body. Keep it simple: inline styles, no scripts, no remote images you don't control.
<code>replyTo</code>	no	Where replies go. Defaults to <code>from</code> .

Limits

Five recipients per request. To reach more people, make more requests. A bulk endpoint is planned; it is not available yet, and a request with more than five recipients is refused with a 400 rather than partly sent.

Attachments are not supported in this version.

4. Responses

Every response is JSON. Read the status code first, then the body.

Status	Meaning	Body
202 Accepted	TatvaOS has the message and will deliver it. This is not confirmation of arrival — no mail API can promise that. The receiving server may still defer, reject, or file it as spam.	<code>{"outcome":"accepted", "recipients":1, "note":"Accepted for delivery. This is not confirmation of arrival."}</code>
400 Bad Request	Something in the request. The message says exactly which field — a missing subject, a sender that is not one of your mailboxes, a from outside this key's allowed senders, more than five recipients, an address that does not parse.	<code>{"error":"website@x.com is not a mailbox on this organisation. Create it under Mailboxes, on a domain you have verified."}</code>
401 Unauthorized	The key is missing, malformed, unknown, or revoked. All of these get the same answer on purpose — the response never reveals whether a key once existed.	<code>{"error":"That API key is not valid."}</code>
502 Bad Gateway	TatvaOS's own mail server refused the message. The body carries its exact reason. Most often the sender's domain has lost verification. Nothing was sent; it is safe to retry after fixing the cause.	<code>{"error":"...the mail server's reason...", "outcome":"refused"}</code>

Retries

Retry a **502** after fixing the reason it gives, and a network failure at once. Do not retry a **202** — the message is already on its way, and a retry sends it twice. Do not retry a **400** or **401** unchanged; the same request gets the same answer.

5. Code examples

Each of these sends the same message. Replace the key, and the two addresses.

curl

```
curl -X POST https://core.tatvaos.com/api/v1/mail/send \  
-H "Authorization: Bearer $TATVAOS_MAIL_KEY" \  
-H "Content-Type: application/json" \  
-d '{"from":"website@your-domain.com","to":"customer@example.com","subject":"Thanks for getting in touch",
```

Node.js (18+, built-in fetch)

```
const res = await fetch("https://core.tatvaos.com/api/v1/mail/send", {  
  method: "POST",  
  headers: {  
    "Authorization": `Bearer ${process.env.TATVAOS_MAIL_KEY}`,  
    "Content-Type": "application/json",  
  },  
  body: JSON.stringify({  
    from: "website@your-domain.com",  
    to: "customer@example.com",  
    subject: "Thanks for getting in touch",  
    text: "We received your message and will reply within one working day.",  
  }},  
});  
const body = await res.json();  
if (res.status !== 202) throw new Error(`${res.status}: ${body.error}`);
```

Python (requests)

```
import os, requests

r = requests.post(
    "https://core.tatvaos.com/api/v1/mail/send",
    headers={"Authorization": f"Bearer {os.environ['TATVAOS_MAIL_KEY']}"},
    json={
        "from": "website@your-domain.com",
        "to": "customer@example.com",
        "subject": "Thanks for getting in touch",
        "text": "We received your message and will reply within one working day.",
    },
    timeout=30,
)
if r.status_code != 202:
    raise RuntimeError(f"{r.status_code}: {r.json().get('error')}")
```

PHP (cURL)

```
<?php
$payload = json_encode([
    "from" => "website@your-domain.com",
    "to" => "customer@example.com",
    "subject" => "Thanks for getting in touch",
    "text" => "We received your message and will reply within one working day.",
]);
$ch = curl_init("https://core.tatvaos.com/api/v1/mail/send");
curl_setopt_array($ch, [
    CURLOPT_POST => true,
    CURLOPT_POSTFIELDS => $payload,
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        "Authorization: Bearer " . getenv("TATVAOS_MAIL_KEY"),
        "Content-Type: application/json",
    ],
]);
$body = json_decode(curl_exec($ch), true);
$status = curl_getinfo($ch, CURLINFO_HTTP_CODE);
if ($status !== 202) throw new Exception("$status: " . $body["error"]);
```

A website contact form, end to end

The pattern every integration follows: the visitor submits the form to **your** server; your server validates it, then calls TatvaOS with the key from its environment. The browser never sees the key. Send the notification to your team from `website@`, and set `replyTo` to the visitor's address so a reply from your inbox goes straight back to them.

6. Getting into the inbox

TatvaOS handles the technical side: every message is DKIM-signed for your domain, sent from an IP your SPF record authorises, and passes DMARC. Those are necessary and they are done. They are not sufficient. Whether a message lands in the inbox or in spam is decided by the receiver's reputation model, and reputation is earned by the address, over time, with real mail.

What helps

Send from an address you keep using. Reputation attaches to the address and the domain. A new address has none; its first messages often go to spam regardless of content. That improves as people open and reply.

Write like a person. A greeting, a reason for the message, a signature with a name. One-line messages with subjects like "test" are the shape of spam, and filters know it.

Send both text and html. HTML-only mail scores worse. Keep HTML simple.

Only send to people who expect it. Mail that gets marked as spam by recipients damages the address for everyone. A contact-form confirmation to the person who filled it in is expected; a newsletter to a bought list is not, and this API is not built for it.

Start small. Tens a day, not thousands, until the address has a history.

What a receiver sees

The authentication lines Gmail attached to a message sent through this API on 3 September 2026, unedited. This is what TatvaOS does for you on every message:

```
Authentication-Results: mx.google.com;
    dkim=pass header.i=@techvein.com header.s=tv2026a;
    spf=pass (google.com: domain of website_support@techvein.com
        designates 172.105.57.198 as permitted sender);
    dmarc=pass header.from=techvein.com
Received: from mx.tatvaos.com (mx.tatvaos.com. [172.105.57.198])
    by mx.google.com with ESMTPS (version=TLS1_3 cipher=TLS_AES_256_GCM_SHA384)
```

What "accepted" means

A 202 means TatvaOS's server took the message and will attempt delivery. It says nothing about what the receiving server did with it. If you need to know whether a message reached a specific inbox, the honest answer is that no sender can know that — not TatvaOS, not any provider. What a later version will add is the receiving server's own response (accepted, deferred, bounced) per recipient; see section 8.

7. Security checklist

Do	Don't
Keep the key in a server-side secret store or environment variable.	Embed it in browser code, a mobile app, or a public repository.
Use one key per program, named for what it does, and give it only the sender addresses that program needs.	Share one key across your website, billing, and a partner's system.
Ask the administrator to revoke a key the moment you suspect exposure.	Keep using a key that has been in a chat, a ticket, or a log.
Validate form input on your server before sending.	Let a form visitor choose the to or from address.
Rate-limit your own form to stop abuse.	Assume TatvaOS will rate-limit for you — it does not yet.

8. What is not in this version

Stated so you can plan around it rather than discover it. All of these are designed and scheduled; none is available today.

Not yet	What it will be
Delivery status	Per-recipient outcome from the receiving server — accepted, deferred, bounced with reason — in the admin log and, later, via API.
Bounce handling	Automatic suppression of addresses that hard-bounce, so a bad address is never sent to twice.
Bulk sending	One request, many recipients, expanded server-side. Until then: five per request, more requests.
Rate limits	Per-organisation limits with clear 429 responses. Today there are none; be a good neighbour.
Attachments	Not supported. Link to a file instead.
Open / click tracking	Not planned. TatvaOS does not insert tracking pixels or rewrite links.

Version 1.2 — 9 September 2026. Every request, field and response was exercised against production on 3 September 2026, when this guide was first written, and the sample headers in section 6 are from a message Gmail received that day. The key-restriction behaviour in sections 2 and 4 was corrected on 9 September against the endpoint's implementation: earlier versions said one key could send as any mailbox in the organisation, which was never true of a key with an allowed-sender list.

Questions: your organisation's TatvaOS administrator, who can see the send log for every key.